# LAPORAN PRAKTIKUM
# PEMROGRAMAN BERORIENTASI OBJEK
# MODUL 5 – RELASI ANTAR KELAS




NAMA: ANDI MUHAMMAD ABID JAYA

NIM: 103132400014

KELAS: S1-SD-05-01

**PROGRAM STUDI SAINS DATA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Teori

Dalam modul ini mempelajari beberapa dasar teori dasar sebagai berikut:

1. Asosiasi (Association)

Asosiasi adalah hubungan antara dua kelas yang saling berinteraksi atau menggunakan satu sama lain, tetapi tidak harus memiliki objek dari kelas lain sebagai atribut. Hubungan ini dapat terjadi melalui pemanggilan method atau penggunaan objek sebagai parameter.

Contohnya, pada program cafe, kelas Kasir berhubungan dengan kelas Pesanan melalui method prosesPembayaran(Pesanan pesanan). Kelas Kasir menggunakan objek Pesanan untuk memproses pembayaran tanpa menyimpannya sebagai atribut kelas.

2. Agregasi (Aggregation)

Agregasi adalah hubungan antara dua kelas ketika suatu kelas menampung atau menggunakan objek dari kelas lain, tetapi objek tersebut dapat dibuat dan tetap berdiri sendiri tanpa bergantung pada kelas yang menampungnya.

Contohnya, pada program cafe, kelas Pesanan menampung beberapa objek ItemPesanan menggunakan array. Objek ItemPesanan dibuat terlebih dahulu di kelas Main, kemudian dimasukkan ke dalam Pesanan melalui method tambahItem().

3. Komposisi (Composition)

Komposisi adalah hubungan antara dua kelas yang memiliki keterikatan kuat, yaitu objek dari suatu kelas menjadi bagian dari objek kelas lain. Objek bagian tersebut dibuat di dalam kelas yang memilikinya.

Contohnya, pada program cafe, kelas ItemPesanan memiliki objek RincianHarga yang dibuat menggunakan new di dalam constructor ItemPesanan. Dengan demikian, pembuatan objek RincianHarga menjadi bagian dari proses pembuatan ItemPesanan.

B. Guided

1. Guided 1

```
// nama file: Dosen.java

package abid.guided.guided1;

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    // Setter
    public void setKodeDosen(String kodeDosen) {
        this.kodeDosen = kodeDosen;
    }

    public void setNamaDosen(String namaDosen) {
        this.namaDosen = namaDosen;
    }

    // Getter
    public String getKodeDosen() {
        return this.kodeDosen;
    }

    public String getNamaDosen() {
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
        // ini asosiasi, method milik class Mahasiswa dipanggil di class Dosen,
        // tp objek Mahasiswa tidak menjadi atribut dr class Dosen
        s.setNilai(nilai);
    }

    public int getScore(Mahasiswa s){
        // ini asosiasi, method milik class Mahasiswa dipanggil di class Dosen,
        // tp objek Mahasiswa tidak menjadi atribut dr class Dosen
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();
        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);
```

```
        System.out.println("Nim :" + m.getNim());
        System.out.println("Nama :" + m.getNama());
        System.out.println("Nilai :" + d.getScore(m));
    }
}
```

```
//  nama file: Mahasiswa.java

package abid.guided.guided1;

public class Mahasiswa {
    private String nim;
    private String nama;
    private int nilai;

    public void setNama(String nama) {
        this.nama = nama;
    }

    public void setNim(String nim) {
        this.nim = nim;
    }

    public  void setNilai(int nilai){
        this.nilai = nilai;
    }
    public String getNama() {
        return this.nama;
    }

    public int getNilai() {
        return this.nilai;
    }

    public String getNim() {
        return this.nim;
    }
}
```

Output : nanti jadi satu di file main.

2. Guided 2

```
//   nama file: Jurusan.java

package abid.guided.guided2.agregasi;

public class Jurusan {
    private int jumlahMahasiswa = 0; // jangan static
    private String kodeJurusan, namaJurusan;
    private Mahasiswa[] daftarMahasiswa = new Mahasiswa[32];

    // Constructor
    public Jurusan(String kodeJurusan, String namaJurusan){
        this.kodeJurusan = kodeJurusan;
        this.namaJurusan = namaJurusan;
    }

    // Behavior
    public void addMahasiswa(Mahasiswa m){
        if (jumlahMahasiswa < daftarMahasiswa.length) {
            this.daftarMahasiswa[jumlahMahasiswa] = m;
            jumlahMahasiswa++;
        }
    }

    public void displayMahasiswa(){
        System.out.println("kode jurusan: " + this.kodeJurusan);
        System.out.println("Nama jurusan: " + this.namaJurusan);
        System.out.println("daftar mahasiswa: ");

        for (int i = 0; i < jumlahMahasiswa; i++){
            System.out.println((i+1) + ". " + daftarMahasiswa[i].getNIM() + " " +
daftarMahasiswa[i].getNama());
        }
    }

    // Getter
    public String getKodeJurusan() {
        return this.kodeJurusan;
    }

    public String getNamaJurusan() {
```

```
        return this.namaJurusan;
    }

    // Setter
    public void setKodeJurusan(String kodeJurusan) {
        this.kodeJurusan = kodeJurusan;
    }

    public void setNamaJurusan(String namaJurusan) {
        this.namaJurusan = namaJurusan;
    }
}
```

```
//  nama file: Mahasiswa.java

package abid.guided.guided2.agregasi;

public class Mahasiswa {
    private String NIM, Nama;

    public Mahasiswa(String NIM, String Nama) {
        this.NIM = NIM;
        this.Nama = Nama;
    }

    // getter

    public String getNIM() {
        return NIM;
    }

    public  String getNama(){
        return  Nama;
    }

    // setter

    public  void  setnama(String Nama){
        this.Nama = Nama;
    }
```

```
    public  void  setnNIM(String NIM){
        this.NIM = NIM;
    }
}
```

Output : nanti jadi satu di file main.

3. Guided 3

```
// nama file: CPU.java

package abid.guided.guided3;

public class CPU {
    private String Merk;
    private int Kecepatan;

    public CPU(String m, int k) {
        this.Merk = m;
        this.Kecepatan = k;
    }

    public void DisplaySpecCPU() {
        System.out.println(this.Merk + ", " + this.Kecepatan);
    }

}
```

```
// nama file: Komputer.java

        package abid.guided.guided3;

        public class Komputer {
            private CPU cpu;
            private Monitor monitor;
            private Mouse mouse;
```

```
    public Komputer(String merkCpu, int kecepatanCpu, String merkMonitor,
String merkMouse, String typeMouse) {
        this.cpu = new CPU(merkCpu, kecepatanCpu);
        this.monitor = new Monitor(merkMonitor);
        this.mouse = new Mouse(merkMouse, typeMouse);
    }

    public void displaySpecKomputer() {
        System.out.println("=== Spesifikasi Komputer ===");
        System.out.print("CPU     : ");
        cpu.DisplaySpecCPU();

        System.out.print("Monitor : ");
        monitor.DisplaySpecMonitor();

        System.out.print("Mouse   : ");
        mouse.DisplaySpecMouse();
    }
}
```

```
// nama file: Monitor.java

package abid.guided.guided3;

public class Monitor {
    private String Merk;

    public Monitor(String m) {
        this.Merk = m;
    }

    public void DisplaySpecMonitor() {
        System.out.println(this.Merk);
    }
}
```

```
// nama file: Mouse.java

package abid.guided.guided3;

public class Mouse {
```

```
    private String Merk, Type;

    public Mouse(String m, String t) {
        this.Merk = m;
        this.Type = t;
    }

    public void DisplaySpecMouse() {
        System.out.println(this.Merk + ", " + this.Type);
    }
}
```

```
// nama file: Processor.java

package abid.guided.guided3;

public class Processor {
 private String brand;
    private int kecepatan;

    //constructor
    public Processor(String brand,int kecepatan){
        this.brand = brand;
        this.kecepatan = kecepatan;
    }

    //behaviour
    public void displaySpec(){
        System.out.println(this.brand + ", " + this.kecepatan);
    }
}
```

Output : nanti jadi satu di file main.

4. Main

```
// nama File : Main.java
```

```
package abid;

import abid.guided.guided2.agregasi.Jurusan;
import abid.guided.guided2.agregasi.Mahasiswa;
import abid.guided.guided3.Komputer;

public class Main {
    public static void main(String[] args) {
        runGuided2();
        runGuided3();
    }

    public static void runGuided1() {
    }

    public static void runGuided2() {
        Mahasiswa m1 = new Mahasiswa("103112400148", "Rendhi");
        Mahasiswa m2 = new Mahasiswa("103112400149", "John");

        Jurusan prodi = new Jurusan("SD", "Sains Data");

        prodi.addMahasiswa(m1);
        prodi.addMahasiswa(m2);

        prodi.displayMahasiswa();
    }

    public static void runGuided3() {
        Komputer pc = new Komputer("Intel Core i5", 3200, "LG 24 Inch",
"Logitech", "Wireless");
        pc.displaySpecKomputer();
    }
}
```

Output:

```
n'
kode jurusan: SD
Nama jurusan: Sains Data
daftar mahasiswa:
1. 103112400148 Rendhi
2. 103112400149 John
=== Spesifikasi Komputer ===
CPU     : Intel Core i5, 3200
Monitor : LG 24 Inch
Mouse   : Logitech, Wireless
PS D:\SEMESTER 5\PEMROGRAMAN BERBASIS OBJEK\T
```

C. Unguided

1. Soal Unguided

# Soal Unguided — Modul 5: Pemrograman Berorientasi Objek

Relasi Antar Kelas: Komposisi, Agregasi, dan Asosiasi

---

## Petunjuk Umum

Buatlah program Java berdasarkan cerita berikut. Rancang sendiri struktur program yang diperlukan sebelum menuliskan kode.

- Tentukan sendiri field, constructor, dan method yang diperlukan, mengikuti hint dan tugas yang disediakan.
- Data di dalam object tidak boleh diakses atau diubah langsung dari luar class-nya (gunakan enkapsulasi).
- Gunakan nama parameter yang sama dengan nama field pada constructor (sehingga `this` perlu dipakai secara eksplisit).
- Proyek wajib mengikuti konvensi penamaan standar (PascalCase untuk Class, camelCase untuk variabel/method).
- Gunakan array standar bertipe tetap (bukan `ArrayList` atau koleksi library lainnya).
- Tidak perlu menggunakan inheritance, overriding, maupun abstract class.

---

## Soal — Sistem Manajemen Pesanan Cafe

Sebuah cafe membutuhkan sistem sederhana untuk mencatat dan memproses pesanan pelanggan. Dalam sistem ini, relasi antar kelas dirancang untuk mencerminkan kondisi nyata operasional cafe.

Pengerjaan soal ini dibagi menjadi 3 bagian tugas utama yang masing-masing melatih satu jenis relasi antar kelas, diikuti dengan pengujian pada kelas utama (`Main`).

---

### Tugas 1: Implementasi Komposisi (Composition)

Komposisi adalah relasi di mana sebuah objek memiliki objek lain sebagai bagian internalnya yang **tidak dapat berdiri sendiri**. Siklus hidup objek internal terikat penuh pada objek utamanya.

Pada sistem cafe:

- Setiap item pesanan (`ItemPesanan`) pasti memiliki rincian harga (`RincianHarga`) yang mencatat harga dasar dan pajak (10%).
- Objek `RincianHarga` harus di-instansiasi langsung di dalam constructor `ItemPesanan`. Objek `RincianHarga` tidak pernah dibuat di `main`. Jika `ItemPesanan` dihapus, maka `RincianHarga` miliknya ikut hilang.

#### Kebutuhan Tugas 1

Buatlah dua class: `RincianHarga` dan `ItemPesanan`.

**Field yang disarankan:**

| Class | Field | Jenis | Tipe | Keterangan |
|---|---|---|---|---|
| `RincianHarga` | `hargaDasar` | Instance field | `double` | Harga awal sebelum pajak |
| `RincianHarga` | `pajak` | Instance field | `double` | Nilai nominal pajak (10% dari harga dasar) |
| `ItemPesanan` | `namaItem` | Instance field | `String` | Nama menu (misal: "Iced Americano") |
| `ItemPesanan` | `rincian` | Instance field | `RincianHarga` | Objek internal rincian harga (**Komposisi**) |

**Method yang disarankan:**

| Class | Method | Return | Keterangan |
|---|---|---|---|
| `RincianHarga` | `getHargaTotal()` | `double` | Mengembalikan nilai `hargaDasar + pajak` |
| `ItemPesanan` | `getNamaItem()` | `String` | Mengembalikan nama item pesanan |

| Class | Method | Return | Keterangan |
|---|---|---|---|
| `ItemPesanan` | `getRincian()` | `RincianHarga` | Mengembalikan objek rincian harga |

---

## Tugas 2: Implementasi Agregasi (Aggregation)

Agregasi adalah relasi di mana sebuah objek menampung kumpulan objek lain yang **dapat berdiri sendiri**. Objek yang ditampung dibuat secara independen di luar, lalu dimasukkan ke dalam objek wadah.

Pada sistem cafe:

- Sebuah `Pesanan` menampung beberapa objek `ItemPesanan` menggunakan array standar.
- Objek `ItemPesanan` dibuat terpisah terlebih dahulu (di `main`), lalu dimasukkan satu per satu ke dalam `Pesanan` melalui method `tambahItem()`. Jika `Pesanan` dibatalkan atau dihapus, objek `ItemPesanan` tersebut secara konsep tetap ada.

### Kebutuhan Tugas 2

Buatlah class `Pesanan`.

**Field yang disarankan:**

| Field | Jenis | Tipe | Keterangan |
|---|---|---|---|
| `nomorMeja` | Instance field | `int` | Nomor meja pelanggan |
| `daftarItem` | Instance field | `ItemPesanan[]` | Array penampung item pesanan (**Agregasi**) |
| `jumlahItem` | Instance field | `int` | Penanda jumlah item yang sudah masuk ke array |

**Method yang disarankan:**

| Method | Return | Keterangan |
|---|---|---|
| `tambahItem(ItemPesanan item)` | `void` | Memasukkan objek `ItemPesanan` dari luar ke dalam array `daftarItem` |
| `tampilkanPesanan()` | `void` | Menampilkan nomor meja dan seluruh item beserta harga total per item |
| `getNomorMeja()` | `int` | Mengembalikan nomor meja |
| `getDaftarItem()` | `ItemPesanan[]` | Mengembalikan array daftar item |
| `getJumlahItem()` | `int` | Mengembalikan jumlah item yang tersimpan |

---

## Tugas 3: Implementasi Asosiasi (Association)

Asosiasi adalah relasi interaksi antara dua objek independen yang **saling menggunakan tetapi tidak saling memiliki**. Objek tidak menyimpan objek lain sebagai field permanen, melainkan berinteraksi melalui parameter method.

Pada sistem cafe:

- Seorang `Kasir` memproses pembayaran dari sebuah `Pesanan`.
- Class `Kasir` tidak menyimpan objek `Pesanan` sebagai field, melainkan menerima objek `Pesanan` sebagai parameter pada method `prosesPembayaran()`.

### Kebutuhan Tugas 3

Buatlah class `Kasir`.

**Field yang disarankan:**

| Field | Jenis | Tipe | Keterangan |
|---|---|---|---|
| `namaKasir` | Instance field | `String` | Nama petugas kasir |

**Method yang disarankan:**

| Method | Return | Keterangan |
| --- | --- | --- |
| `prosesPembayaran(Pesanan pesanan)` | `void` | Mengakses data `Pesanan`, menghitung akumulasi total bayar seluruh item, lalu mencetak struk pembayaran (**Asosiasi**) |

---

## Tugas 4: Eksekusi dan Pengujian di `Main`

Buatlah class `Main` untuk merangkai seluruh kelas dan menguji ketiga jenis relasi.

### Alur program di `main`

1. Buat 1 objek `Kasir` (misal: `"Budi"`).
2. Buat 3 objek `ItemPesanan` secara independen:
   - `"Iced Americano"`, harga dasar: `18000`
   - `"Beef Croissant"`, harga dasar: `25000`
   - `"Matcha Latte"`, harga dasar: `22000`
3. Buat 1 objek `Pesanan` untuk **Meja 4** (kapasitas array misal: 5 item).
4. Masukkan ketiga objek `ItemPesanan` ke dalam objek `Pesanan` menggunakan method `tambahItem()`.
5. Tampilkan rincian pesanan dengan memanggil `tampilkanPesanan()` pada objek `Pesanan`.
6. Jalankan pemrosesan pembayaran dengan memanggil method `prosesPembayaran(pesanan)` dari objek `Kasir`.

---

## Output yang Diharapkan

```
None
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
```

```
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
==============================
```

---

## Yang Perlu Diperhatikan Saat Mengerjakan

- **Komposisi:** Objek `RincianHarga` wajib di-instansiasi di dalam constructor `ItemPesanan` menggunakan kata kunci `new`. Dilarang melakukan `new RincianHarga(...)` di dalam `main`.
- **Agregasi:** Objek `ItemPesanan` wajib di-instansiasi di `main` terlebih dahulu, kemudian dikirimkan sebagai argumen ke method `tambahItem()`.
- **Asosiasi:** Class `Kasir` dilarang memiliki field bertipe `Pesanan`. Interaksi hanya terjadi saat method `prosesPembayaran(Pesanan pesanan)` dipanggil.

Jawaban :

```
// nama file: ItemPesanan.java


package abid.unguided;

public class ItemPesanan {
    private String namaItem;
    private RincianHarga rincian;

    public ItemPesanan(String namaItem, double hargaDasar) {
        this.namaItem = namaItem;
        this.rincian = new RincianHarga(hargaDasar);
    }

    public String getNamaItem() {
        return this.namaItem;
    }

    public RincianHarga getRincian() {
        return this.rincian;
    }
}
```

```
// nama file: RincianHarga.java

        package abid.unguided;

        public class RincianHarga {
            private double hargaDasar;
            private double pajak;

            public RincianHarga(double hargaDasar) {
                this.hargaDasar = hargaDasar;
                this.pajak = hargaDasar * 0.10;
            }

            public double getHargaDasar() {
                return this.hargaDasar;
            }
```

```
    public double getPajak() {
        return this.pajak;
    }

    public double getHargaTotal() {
        return this.hargaDasar + this.pajak;
    }
}
```

```
// nama file: Pesanan.java

package abid.unguided;

public class Pesanan {
    private int nomorMeja;
    private ItemPesanan[] daftarItem;
    private int jumlahItem;

    public Pesanan(int nomorMeja, int kapasitas) {
        this.nomorMeja = nomorMeja;
        this.daftarItem = new ItemPesanan[kapasitas];
        this.jumlahItem = 0;
    }

    public void tambahItem(ItemPesanan item) {
        if (this.jumlahItem < this.daftarItem.length) {
            this.daftarItem[this.jumlahItem] = item;
            this.jumlahItem++;
        } else {
            System.out.println("Kapasitas pesanan sudah penuh.");
        }
    }

    public void tampilkanPesanan() {
        System.out.println("=== DAFTAR PESANAN CAFE ===");
        System.out.println("Nomor Meja: " + this.nomorMeja);

        for (int i = 0; i < this.jumlahItem; i++) {
            ItemPesanan item = this.daftarItem[i];
            RincianHarga rincian = item.getRincian();
```

```
                System.out.println(
                    (i + 1) + ". " + item.getNamaItem()
                    + " - Rp " + rincian.getHargaTotal()
                    + " (Harga: " + rincian.getHargaDasar()
                    + ", Pajak: " + rincian.getPajak() + ")"
                );
            }
        }

        public int getNomorMeja() {
            return this.nomorMeja;
        }

        public ItemPesanan[] getDaftarItem() {
            return this.daftarItem.clone();
        }

        public int getJumlahItem() {
            return this.jumlahItem;
        }
    }
```

```
// nama file: Kasir.java

    package abid.unguided;

    public class Kasir {
        private String namaKasir;

        public Kasir(String namaKasir) {
            this.namaKasir = namaKasir;
        }

        public void prosesPembayaran(Pesanan pesanan) {
            double totalSubtotal = 0;
            double totalPajak = 0;

            ItemPesanan[] daftarItem = pesanan.getDaftarItem();
            int jumlahItem = pesanan.getJumlahItem();
```

```
        for (int i = 0; i < jumlahItem; i++) {
            RincianHarga rincian = daftarItem[i].getRincian();

            totalSubtotal += rincian.getHargaDasar();
            totalPajak += rincian.getPajak();
        }

        double totalBayar = totalSubtotal + totalPajak;

        System.out.println();
        System.out.println("=== STRUK PEMBAYARAN KASIR ===");
        System.out.println("Kasir Bertugas : " + this.namaKasir);
        System.out.println("Pesanan        : Meja " + pesanan.getNomorMeja());
        System.out.println("Jumlah Item    : " + jumlahItem);
        System.out.println("Total Subtotal : Rp " + totalSubtotal);
        System.out.println("Total Pajak    : Rp " + totalPajak);
        System.out.println("TOTAL BAYAR    : Rp " + totalBayar);
        System.out.println("==============================");
    }
}
```

```
// nama file: Main.java

package abid.unguided;

import abid.unguided.ItemPesanan;
import abid.unguided.Kasir;
import abid.unguided.Pesanan;
import abid.unguided.RincianHarga;

public class Main {
    public static void main(String[] args) {
        // Membuat objek Kasir
        Kasir kasir = new Kasir("Budi");

        // Membuat objek ItemPesanan secara independen
        ItemPesanan item1 = new ItemPesanan(
            "Iced Americano", 18000
        );
```

```
        ItemPesanan item2 = new ItemPesanan(
            "Beef Croissant", 25000
        );

        ItemPesanan item3 = new ItemPesanan(
            "Matcha Latte", 22000
        );

        // Membuat objek Pesanan untuk meja 4
        Pesanan pesanan = new Pesanan(4, 5);

        // Memasukkan item ke dalam Pesanan (Agregasi)
        pesanan.tambahItem(item1);
        pesanan.tambahItem(item2);
        pesanan.tambahItem(item3);

        // Menampilkan daftar pesanan
        pesanan.tampilkanPesanan();

        // Memproses pembayaran (Asosiasi)
        kasir.prosesPembayaran(pesanan);
    }
}
```

Output :

```
\modul5\target\classes' 'abid.unguided.Main'
=== DAFTAR PESANAN CAFE ===
Nomor Meja: 4
1. Iced Americano - Rp 19800.0 (Harga: 18000.0, Pajak: 1800.0)
2. Beef Croissant - Rp 27500.0 (Harga: 25000.0, Pajak: 2500.0)
3. Matcha Latte - Rp 24200.0 (Harga: 22000.0, Pajak: 2200.0)

=== STRUK PEMBAYARAN KASIR ===
Kasir Bertugas : Budi
Pesanan        : Meja 4
Jumlah Item    : 3
Total Subtotal : Rp 65000.0
Total Pajak    : Rp 6500.0
TOTAL BAYAR    : Rp 71500.0
============================
PS D:\SEMESTER 5\PEMROGRAMAN BERBASIS OBJEK\TEMPLATE REPO\praktikum-pbo-ps1ds-05-reg-01>
```

Penjelasan :

File RincianHarga.java berisi kelas RincianHarga dengan field private hargaDasar dan pajak untuk menerapkan enkapsulasi. Constructor digunakan untuk menginisialisasi harga dasar dan menghitung pajak sebesar 10%. Method getHargaDasar(), getPajak(), dan getHargaTotal() digunakan untuk mengambil harga dasar, pajak, dan total harga.

File ItemPesanan.java berisi kelas ItemPesanan dengan field namaItem dan rincian. Constructor membuat objek RincianHarga menggunakan new di dalam constructor ItemPesanan. Hal ini menerapkan relasi komposisi karena rincian harga menjadi bagian internal dari item pesanan. Method getNamaItem() dan getRincian() digunakan untuk mengambil data item dan rincian harganya.

File Pesanan.java berisi kelas Pesanan dengan field nomorMeja, daftarItem, dan jumlahItem. Kelas ini menggunakan array ItemPesanan[] untuk menampung item. Method tambahItem() digunakan untuk memasukkan objek item yang telah dibuat secara terpisah, sehingga menerapkan relasi agregasi. Method tampilkanPesanan() menampilkan daftar item beserta harga dan pajaknya, sedangkan getter digunakan untuk mengambil data pesanan.

File Kasir.java berisi kelas Kasir dengan field namaKasir. Method prosesPembayaran(Pesanan pesanan) menerima objek Pesanan sebagai parameter, menghitung subtotal, total pajak, dan total pembayaran, lalu mencetak struk. Hal ini menerapkan relasi asosiasi karena objek Pesanan hanya digunakan melalui parameter method dan tidak disimpan sebagai field di kelas Kasir.

File Main.java berisi method main() untuk menjalankan program. Program membuat satu objek Kasir, tiga objek ItemPesanan, dan satu objek Pesanan untuk meja 4. Ketiga item dimasukkan ke pesanan menggunakan tambahItem(), kemudian ditampilkan melalui tampilkanPesanan() dan diproses pembayarannya melalui prosesPembayaran(). Hasil akhirnya adalah subtotal Rp65.000, pajak Rp6.500, dan total bayar Rp71.500.

## D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan konsep enkapsulasi sangat penting untuk melindungi data pada setiap kelas dengan menggunakan access modifier private dan method getter. Praktikum ini juga menunjukkan penerapan komposisi pada kelas ItemPesanan dan RincianHarga, yaitu objek RincianHarga dibuat langsung di dalam constructor ItemPesanan sebagai bagian internalnya. Selain itu, agregasi diterapkan pada kelas Pesanan dan ItemPesanan, di mana objek ItemPesanan dibuat secara independen terlebih dahulu, kemudian dimasukkan ke dalam array menggunakan method tambahItem().

Praktikum ini juga membuktikan penerapan asosiasi pada kelas Kasir dan Pesanan, yaitu melalui method prosesPembayaran() yang menerima objek Pesanan sebagai parameter tanpa menyimpannya sebagai field kelas. Penggunaan array standar memungkinkan kelas Pesanan menampung beberapa item dengan kapasitas tetap, sedangkan pembagian program ke dalam beberapa kelas membuat kode lebih terstruktur dan mudah dipahami. Dengan demikian, praktikum ini membantu memahami perbedaan komposisi, agregasi, dan asosiasi serta penerapannya dalam pemrograman berorientasi objek menggunakan Java.